

Wybrane zagadnienia Algebry 2026

Sprawozdanie 2.

Sara Żyndul

279686

25 czerwca 2026

1 Laboratorium 4: Wielomiany wielu zmiennych i bazy Gröbnera

1.1 Ad a) Pierścienie wielomianów wielu zmiennych i różnice w implementacji

Wielomiany wielu zmiennych w pierścieniu $\mathbb{R}[x_1, \dots, x_n]$ zostały zaimplementowane w oparciu o strukturę słownikową (tzw. reprezentację rzadką). Każdy wielomian reprezentowany jest jako obiekt, w którym:

- **Klucz słownika** (dictionary key) stanowi wektor potęg poszczególnych zmiennych zapisany w postaci niemutowalnej krotki (`tuple`). Na przykład w pierścieniu $\mathbb{R}[x, y, z]$ krotka $(2, 7, 0)$ jednoznacznie odpowiada jednomianowi x^2y^7 .
- **Wartość słownika** (dictionary value) odpowiada niezerowemu współczynnikowi rzeczywistemu (użytemu jako klasa `Fraction`) stojącemu przy danym jednomianie.

Różnice w implementacji względem wielomianów jednej zmiennej:

1. **Struktura danych:** W przypadku jednej zmiennej jednomiany można wydajnie reprezentować za pomocą zwykłej listy, gdzie indeks elementu bezpośrednio wyznacza potęgę zmiennej (reprezentacja gęsta). Dla wielu zmiennych liczba potencjalnych kombinacji wykładników rośnie wykładniczo, co przy reprezentacji gęstej prowadziłoby do wybuchu pamięciowego. Reprezentacja rzadka (słownikowa) przechowuje wyłącznie niezerowe składniki.
2. **Brak naturalnego porządku:** Wielomiany jednej zmiennej posiadają naturalny, jednoznaczny i liniowy porządek jednomianów wyznaczany przez ich stopień ($x^0 < x^1 < x^2 < \dots$). Wyznaczenie wyrazu wiodącego (*Leading Term*, LT) jest tam trywialne. W przypadku wielu zmiennych nie istnieje jeden naturalny porządek (porównanie jednomianów x^2 oraz y^3 zależy wyłącznie od przyjętej konwencji). Z tego powodu klasa musi jawnie przyjmować ciąg zmiennych (`vars`) oraz kryterium porządku (np. `order_vars`), na podstawie którego dynamicznie mapuje krotki wykładników za pomocą dedykowanych funkcji klucza (`lex_key`).

1.2 Ad b) Implementacja operacji redukcji, syzygium oraz algorytmu Buchbergera

W celu generowania baz Gröbnera zaimplementowano trzy kluczowe operacje algebraiczne:

- **PolynomialReduce(f, G):** Realizuje zachłanny algorytm dzielenia wielomianu f przez ciąg wielomianów $\mathcal{G} = (g_1, \dots, g_k)$. W każdej iteracji algorytm sprawdza, czy wyraz wiodący aktualnego wielomianu p jest podzielny przez wyraz wiodący któregoś z dzielników g_i . Jeśli tak, dokonuje redukcji; w przeciwnym wypadku przenosi wyraz wiodący do reszty r .
- **Syzygy(f, g):** Wyznacza S-wielomian (syzygium) dla pary wielomianów f i g . Operacja ta polega na znalezieniu najmniejszej wspólnej wielokrotności (LCM) ich wykładników wiodących, a następnie ich wzajemnym skróceniu (eliminacji najwyższych składników):

$$S(f, g) = \frac{x^\gamma}{LT(f)} \cdot f - \frac{x^\gamma}{LT(g)} \cdot g, \quad \text{gdzie } x^\gamma = \text{lcm}(LM(f), LM(g)) \quad (1)$$

- **Buchberger(G_init):** Konstruuje bazę Gröbnera poprzez sukcesywne badanie redukcji S-wielomianów dla wszystkich par wielomianów znajdujących się w bazie. Jeśli reszta z redukcji $S(g_i, g_j)$ przez aktualną bazę jest niezerowa, jest ona dopisywana do zbioru generatorów. Proces kończy się, gdy wszystkie pary redukują się do zera. Na koniec baza zostaje zminimalizowana i w pełni zredukowana.

Poniżej przedstawiono kompletną implementację tych algorytmów w języku Python:

```
1 def polynomial_reduce(f, G, order_vars=None):
2     if order_vars is None:
3         order_vars = f.vars
4     vars_list = f.vars
5
6     r = Polynomial({}, vars_list)
7     p = Polynomial(f.terms, vars_list)
8
9     while not p.is_zero():
10        lt_p_exps, lt_p_coeff = p.leading_term(order_vars)
11        reduced = False
12        for g in G:
13            if g.is_zero():
14                continue
15            lt_g_exps, lt_g_coeff = g.leading_term(order_vars)
16            div = divide_monomials(lt_p_exps, lt_g_exps)
17            if div is not None:
18                # p = p - (lt_p / lt_g) * g
19                mono_dict = {div: lt_p_coeff / lt_g_coeff}
20                monomial_poly = Polynomial(mono_dict, vars_list)
21                p = p - monomial_poly * g
22                reduced = True
23                break
24        if not reduced:
25            # lt_p nie jest redukowalny przez zadne g_i -> trafia do
26            reszty
27            r = r + Polynomial({lt_p_exps: lt_p_coeff}, vars_list)
```

```

27         p = p - Polynomial({lt_p_exps: lt_p_coeff}, vars_list)
28     return r
29
30 def syzygy(f, g, order_vars=None):
31     if f.is_zero() or g.is_zero():
32         return Polynomial({}, f.vars)
33     lt_f_exp, lt_f_coeff = f.leading_term(order_vars)
34     lt_g_exp, lt_g_coeff = g.leading_term(order_vars)
35
36     lcm_exp = lcm_monomials(lt_f_exp, lt_g_exp)
37
38     div_f = divide_monomials(lcm_exp, lt_f_exp)
39     div_g = divide_monomials(lcm_exp, lt_g_exp)
40
41     S_f = Polynomial({div_f: Fraction(1, lt_f_coeff)}, f.vars) * f
42     S_g = Polynomial({div_g: Fraction(1, lt_g_coeff)}, g.vars) * g
43     return S_f - S_g
44
45 def buchberger(G_init, order_vars=None):
46     G = [g for g in G_init if not g.is_zero()]
47     pairs = [(G[i], G[j]) for i in range(len(G)) for j in range(i+1, len
48 (G))]
49
50     while pairs:
51         g1, g2 = pairs.pop(0)
52         S = syzygy(g1, g2, order_vars)
53         rem = polynomial_reduce(S, G, order_vars)
54         if not rem.is_zero():
55             for g in G:
56                 pairs.append((g, rem))
57             G.append(rem)
58
59     # Minimalizacja i redukcja bazy Groebnera
60     cleaned_G = []
61     for g in G:
62         lt_exp, lt_coeff = g.leading_term(order_vars)
63         zero_tuple = (0,) * len(g.vars)
64         cleaned_G.append(g * Polynomial({zero_tuple: Fraction(1,
65 lt_coeff)}, g.vars))
66
67     reduced_G = []
68     for i in range(len(cleaned_G)):
69         other_G = cleaned_G[:i] + cleaned_G[i+1:]
70         rem = polynomial_reduce(cleaned_G[i], other_G, order_vars)
71         if not rem.is_zero():
72             lt_exp, lt_coeff = rem.leading_term(order_vars)
73             zero_tuple = (0,) * len(rem.vars)
74             rem = rem * Polynomial({zero_tuple: Fraction(1, lt_coeff)},
75 rem.vars)
76             if rem.terms not in [r.terms for r in reduced_G]:
77                 reduced_G.append(rem)
78
79     return reduced_G

```

Listing 1: Kody źródłowe funkcji algorytmu Buchbergera

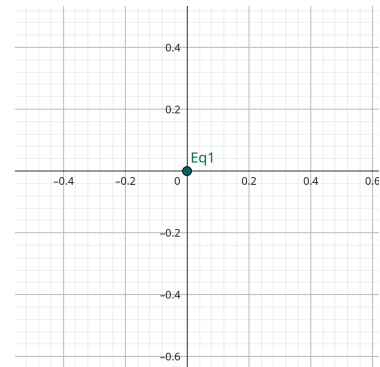
1.3 Ad c) Analiza układu stożka i płaszczyzn (eliminacja zmiennych)

Rozważamy układ równań składający się ze stożka $x^2 + y^2 = z^2$ oraz płaszczyzn $f(x, y, z) = 0$. W celu eliminacji zmiennej z z układu i wyznaczenia ideału głównego $\mathcal{I}_f$ w pierścieniu $\mathbb{R}[x, y]$, wygenerowano bazy Gröbnera przy użyciu porządku leksykograficznego $z > x > y$.

Wielomian niezawierający zmiennej z stanowi poszukiwany generator ideału. Poniżej przedstawiono wyniki działania zaimplementowanego algorytmu Buchbergera dla stałych z numeru indeksu ($a = 2, c = 9, d = 6, e = 8, f = 6$) wraz z rzutami geometrycznymi otrzymanych rozmaitości $V(\mathcal{I}_f)$ na płaszczyznę XY (tzw. krzywych stożkowych).

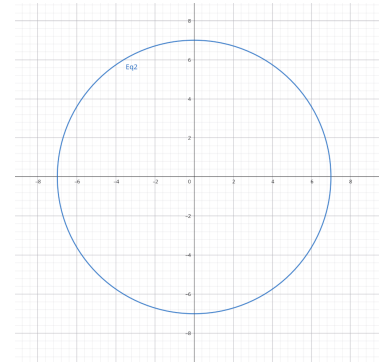
• **Przypadek 1:** $f(x, y, z) = (c + 1)z = 10z$

- **Ideał główny:** $\mathcal{I}_{f_1} = \langle x^2 + y^2 \rangle$
- **Rozmaitość:** Pojedynczy punkt $(0, 0)$.
- **Uzasadnienie geometryczne:** Płaszczyzna $z = 0$ przecina stożek dokładnie w jego wierzchołku, nie obejmując żadnych innych punktów.



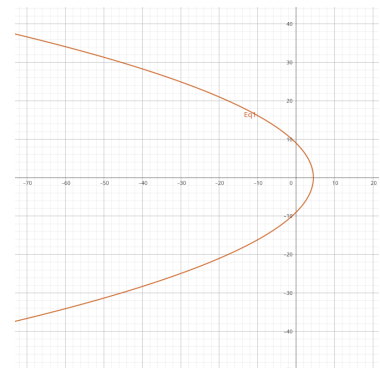
• **Przypadek 2:** $f(x, y, z) = z + d + 1 = z + 7$

- **Ideał główny:** $\mathcal{I}_{f_2} = \langle x^2 + y^2 - 49 \rangle$
- **Rozmaitość:** Okrąg o promieniu $R = 7$.
- **Uzasadnienie geometryczne:** Płaszczyzna $z = -7$ jest prostopadła do pionowej osi stożka. Poziome cięcie stożka obrotowego zawsze generuje okrąg.



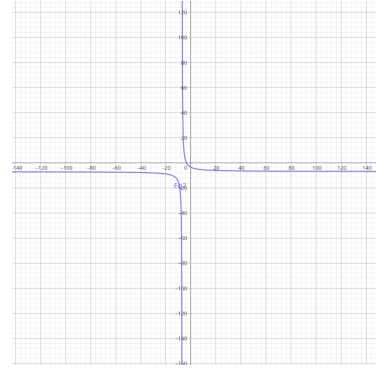
• **Przypadek 3:** $f(x, y, z) = x + z - e - 1 = x + z - 9$

- **Ideał główny:** $\mathcal{I}_{f_3} = \langle x + \frac{1}{18}y^2 - \frac{9}{2} \rangle$
- **Rozmaitość:** Parabola.
- **Uzasadnienie geometryczne:** Płaszczyzna tnąca jest nachylona do osi stożka pod tym samym kątem, co jego boczna tworząca. Skutkuje to powstaniem otwartej krzywej parabolicznej.



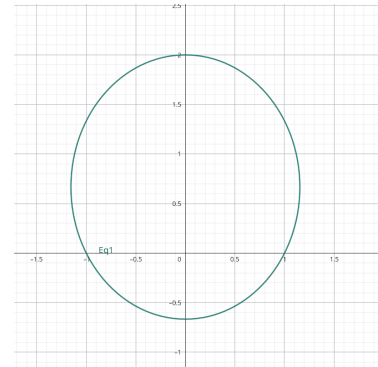
• **Przypadek 4:** $f(x, y, z) = x + y + z + f + 1 = x + y + z + 7$

- **Ideał główny:** $\mathcal{I}_{f_4} = \langle xy + 7x + 7y + \frac{49}{2} \rangle$
- **Rozmaitość:** Hiperbola.
- **Uzasadnienie geometryczne:** Stromo nachylona płaszczyzna przecina obie połą- cie stożka (górną i dolną), tworząc krzywą składającą się z dwóch oddzielnych gałęzi.



• **Przypadek 5:** $f(x, y, z) = \frac{y}{a} + z + 1 = \frac{1}{2}y + z + 1$

- **Ideał główny:** $\mathcal{I}_{f_5} = \langle x^2 + \frac{3}{4}y^2 - y - 1 \rangle$
- **Rozmaitość:** Elipsa.
- **Uzasadnienie geometryczne:** Płaszczyzna przecina stożek na wskroś, obejmując tylko jedną połąć, ale pod kątem mniejszym niż nachylenie tworzącej. Generuje to krzywą zamkniętą. Dodatnie i różne współczynniki przy x^2 i y^2 potwierdzają kształt elipsy.



1.4 Ad d) Eliminacja zmiennej x oraz analiza przekroju kardio- idalnego

W tym punkcie poddano analizie układ równań wyższego stopnia, w którym pierwsze równanie definiuje powierzchnię stożka kardioidalnego, a drugie płaszczyznę tnącą:

$$\begin{cases} (x^2 + y^2 - ax)^2 = z^2(x^2 + y^2) \\ x + 2y + 3z = 0 \end{cases} \quad (2)$$

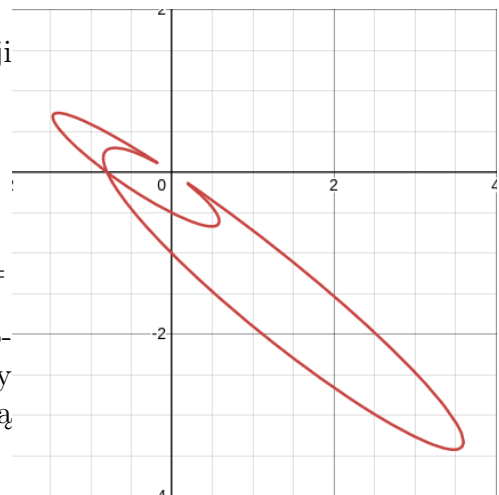
Dla wartości parametru $a = 2$ wyznaczono bazę Gröbnera przy porządku leksykograficznym $x > y > z$, co pozwoliło na całkowite wyeliminowanie zmiennej x i uzyskanie generatora ideału w pierścieniu $\mathbb{R}[y, z]$.

• **Wykres 1 (Rzut rozwiązania układu na płaszczyznę YZ):**

Wyselekcjonowany z bazy wielomian eliminacji zmiennej x przyjmuje postać:

$$y^4 + \frac{24}{5}y^3z + \frac{8}{5}y^3 + \frac{229}{25}y^2z^2 + \frac{156}{25}y^2z + \frac{16}{25}y^2 + \frac{204}{25}yz^3 + \frac{216}{25}yz^2 + \frac{48}{25}yz + \frac{72}{25}z^4 + \frac{108}{25}z^3 + \frac{36}{25}z^2 = 0$$

Generuje on płaską krzywą algebraiczną stopnia 4 (kwartykę), przedstawiającą geometryczny rzut linii przecięcia płaszczyzny z powierzchnią na płaszczyznę współrzędnych yz .



- **Wykres 2 (Przekrój powierzchni dla $z = a = 2$):**

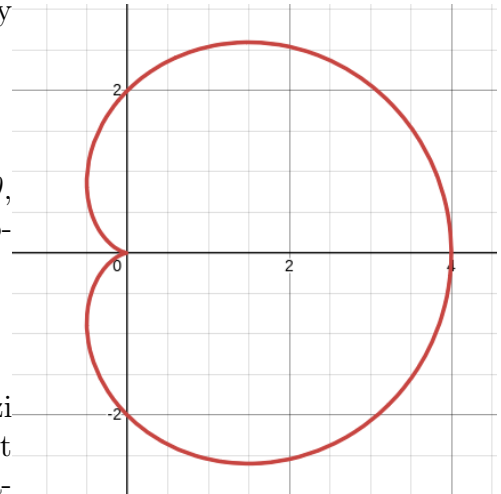
Rozważając wyłącznie pierwsze równanie wyjściowe dla stałej wysokości $z = 2$, otrzymujemy relację:

$$(x^2 + y^2 - 2x)^2 = 4(x^2 + y^2)$$

Przejdźcie do układu biegunowego ($x = r \cos \theta$, $y = r \sin \theta$) pozwala uprościć to wyrażenie do postaci:

$$(r^2 - 2r \cos \theta)^2 = 4r^2 \implies r = 2(1 + \cos \theta)$$

Krzywa ta to klasyczna **kardioida**. Dowodzi to, że badana powierzchnia trójwymiarowa jest strukturą, której poziome przekroje są skalowanymi i przesuniętymi kardioidami.



1.5 Ad e) Wyznaczenie równania kartezjańskiego i analiza otrzymanej krzywej

Przedmiotem analizy jest krzywa zdefiniowana w układzie biegunowym równaniem podanym w zadaniu:

$$r = b \left(4 \cos(\theta) + \frac{1}{\cos(\theta)} \right) \quad (3)$$

Dla przydzielonej wartości parametru $b = 7$ równanie to przybiera postać $r = 28 \cos(\theta) + \frac{7}{\cos(\theta)}$. Wykorzystując związki transformacji biegunowej ($x = r \cos(\theta)$, $y = r \sin(\theta)$, $r^2 = x^2 + y^2$) oraz zaimplementowany algorytm baz Gröbnera do eliminacji zmiennych, wyznaczono jawne równanie algebraiczne w układzie kartezjańskim.

- **Postać wielomianowa:**

$$x^3 + xy^2 - 35x^2 - 7y^2 = 0$$

- **Własności geometryczne:** Po przekształceniu równania do postaci $y^2 = \frac{x^2(35-x)}{x-7}$ łatwo zauważyć, że ciągła gałąź krzywej istnieje wyłącznie dla domeny $x \in (7, 35]$. Z lewej strony ogranicza ją asymptota pionowa $x = 7$, a z prawej wierzchołek w punkcie $(35, 0)$.

- **Punkt izolowany:** Z matematycznego punktu widzenia, równanie jest spełnione również dla punktu $(0, 0)$. Jest to jednak punkt izolowany (odosobniony), którego standardowe narzędzia graficzne nie renderują, stąd krzywa wizualnie omija początek układu współrzędnych.

- **Uwaga do trysektrisy:** Klasyczna trysektrisa Maclaurina (służąca historycznie do geometrycznego podziału kąta na trzy części) posiada w swoim równaniu biegunowym znak minus. Zastosowanie znaku plus (zgodnie z treścią zadania) spowodowało "otwarcie" charakterystycznej pętli i powstanie krzywej o zbadanych wyżej właściwościach.

